

Assignment 1: Neural Networks

Summary Report: Tuning a Neural Network to Classify IMDB Movie Reviews

Shujath Ali Ansari Mohammed | Advanced Machine Learning (64061) | September 2026

Code and knitted output: [Github](#)

Executive Summary

The business question. A neural network was built in class that reads a movie review and predicts whether it is positive or negative. This project asked a practical question: **which design choices make that model more accurate, and which ones are not worth the effort?** I tested 18 versions of the model, changing one design choice at a time, and trained each version three times to separate real improvements from random luck.

88.82% Class model: validation accuracy	89.04% Best model: validation accuracy	+0.22 pp Improvement over class model	≈ 88.3% Accuracy on 25,000 unseen reviews
---	--	---	---

Key findings

- 1. The class model was already close to the best this approach can do.** Every version landed between 88.7% and 89.1% accuracy, so roughly 89 out of 100 reviews are classified correctly whatever the settings.
- 2. The single most useful change was making each layer moderately wider** (32 “neurons” per layer instead of 16). This gave the best result of the study: **89.04%**.
- 3. Adding more layers, using the older “tanh” setting, or switching the error measure to “mse” did not help.** Their effects were smaller than normal run-to-run variation.
- 4. The main risk is over-training.** After about 4 passes through the data, every model starts memorising the training reviews and gets worse on new ones. Stopping training at the right moment matters more than any design change.
- 5. Regularization (dropout and L2)** did not raise the peak score much, but it made the models more stable and consistently lifted accuracy on unseen reviews by about 0.3 points for the class-sized model.

Recommendation

Use a **two-layer network with 32 neurons per layer**, the standard “relu” setting and “binary cross-entropy” error measure, **stop training automatically at the best point** (about 4 passes), and add **dropout** when using larger networks. Do not spend more effort tuning this model type: to go meaningfully beyond 89%, the next investment should be a model that reads **word order** (see Section 6).

1. Background: What the Model Does

The IMDB dataset contains **50,000 movie reviews**, each labelled positive or negative, split evenly into 25,000 reviews for building the model and 25,000 for final testing. Exactly half the reviews are positive, so random guessing would score 50%. The model looks only at **which of the 10,000 most common words appear** in a review (for example “excellent”, “boring”, “waste”) and ignores their order. A neural network is a stack of layers of simple calculating units (“neurons”) that learns from examples which word patterns signal a positive or negative review.

To judge the model fairly, the 25,000 building reviews were split: **15,000 were used for training** (the model learns from these) and **10,000 for validation** (a practice exam used to compare versions and choose the best one). The

25,000 **test** reviews were locked away and used only once at the end, as the final exam. Table 1 explains the terms used in the rest of the report.

Table 1. *Key terms in plain language*

Term	Meaning
Accuracy	Share of reviews the model labels correctly (89% = 89 of every 100 reviews).
Validation accuracy	Accuracy on the 10,000-review practice exam; used to choose the best design.
Test accuracy	Accuracy on 25,000 reviews never seen during design; the honest final score.
Percentage point (pp)	Difference between two percentages: 88.82% → 89.04% is +0.22 pp.
Hidden layer / neurons (units)	The internal calculating layers of the network and how many units each has (its “width”).
Epoch	One full pass of the model through the training reviews.
Overfitting	The model memorises the training reviews instead of learning general patterns, so it gets worse on new reviews.
Activation (relu, tanh)	The mathematical rule each neuron uses to turn its input into an output.
Loss function (binary cross-entropy, mse)	The error measure the model tries to reduce while learning.
Regularization (dropout, L2)	Techniques that stop the model from memorising: dropout randomly switches off neurons during training; L2 penalises extreme weights.
Spread (±) / seed	Each version was trained 3 times from different random starting points (seeds); the spread shows how much results vary by chance.

2. How the Experiments Were Run

The starting point was the model from class: **two hidden layers of 16 neurons, relu activation and binary cross-entropy loss**. I then created 17 variations. Each variation changes **only one thing** compared with the class model, so any change in accuracy can be traced to that one decision. The final group combined the most promising ideas.

Table 2. *The 18 model versions tested*

Group	What was changed	Versions tested
Baseline	nothing (class model)	2 layers × 16 neurons, relu, binary cross-entropy
A. Depth	number of hidden layers	1, 2 (baseline) or 3 layers
B. Width	neurons per layer	8, 16 (baseline), 32, 64, 128
C. Loss function	error measure	binary cross-entropy (baseline) vs mse
D. Activation	neuron rule	relu (baseline) vs tanh
E. Regularization	anti-memorising techniques	L2 (0.001, 0.0001), dropout (0.3, 0.5), L2 + dropout
F. Combined	best ideas together	32 or 64 neurons + dropout 0.5, with and without L2

Four rules kept the comparison fair:

- **Same conditions for every version:** same data, same optimizer (RMSprop), same batch size (512) and up to 20 epochs.
- **Three repeats per version:** results change slightly from run to run by chance, so every version was trained three times and averaged. Differences smaller than **0.25 pp** (twice the typical spread) are treated as “no real difference”.

- **Best stopping point for each version:** because versions over-train at different speeds, each run kept the model from its best epoch on the practice exam.
- **Test data untouched until the end:** the final model was chosen on validation accuracy only, then retrained on all 25,000 building reviews and scored once on the test reviews, alongside the class model for a like-for-like comparison.

3. Findings

3.1 The overall picture

Figure 1 summarises all 18 versions. Each dot is a version's average validation accuracy, the horizontal bar shows its run-to-run spread, and the dashed line marks the class model. **Almost every dot sits within a few tenths of a point of the dashed line, and most bars overlap it:** the class model is already near the ceiling. The clearest gains are in the width group, where the pink dot marks the selected final model (32 neurons).

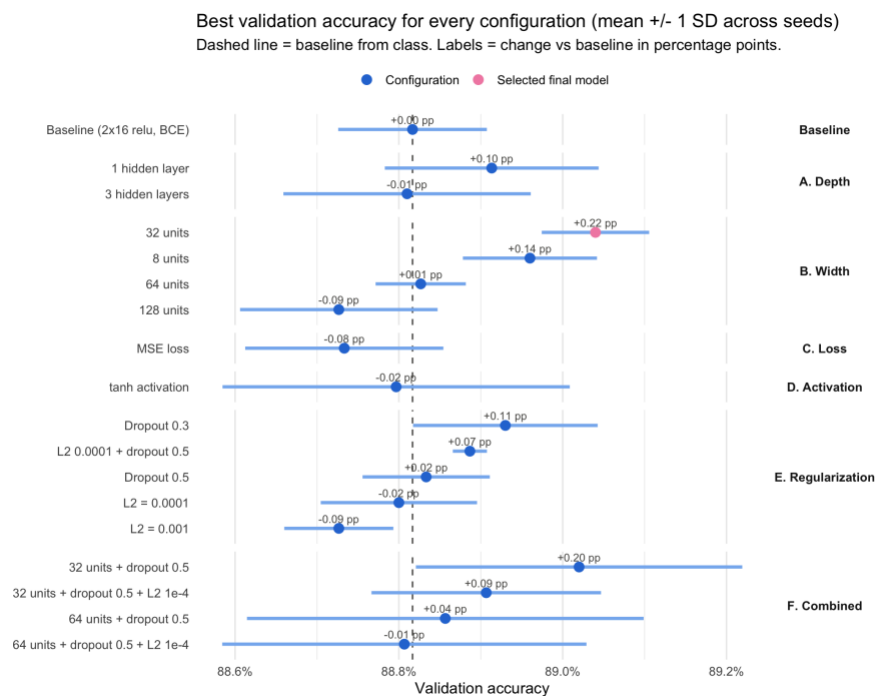


Figure 1. Validation accuracy of all 18 versions (average $\pm$ spread over 3 runs). Dashed line = class model; labels = change in percentage points.

3.2 The main risk: over-training

Figure 2 shows how the class model's error changes as training continues. The **blue line** (error on the training reviews) keeps falling, but the **pink line** (error on the practice exam) is lowest after about **4 epochs** and then rises steadily. From that point the model is memorising the training reviews rather than learning patterns that carry over to new reviews. This pattern appeared in every version tested, which is why each version was scored at its best stopping point.

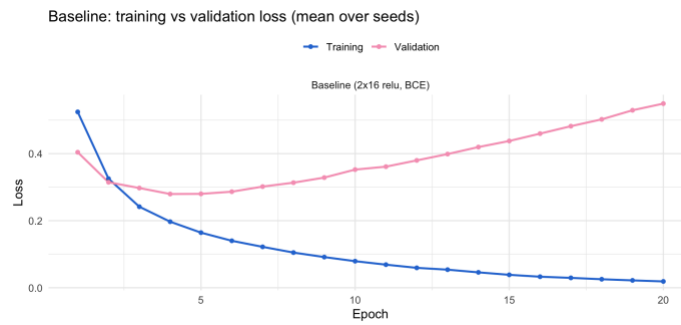


Figure 2. Class model: error on training reviews (blue) vs practice-exam reviews (pink). The practice-exam error bottoms out at epoch 4.

3.3 Number of hidden layers

Table 3. Effect of the number of hidden layers (16 neurons each)

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
1 hidden layer	88.91%	0.13	+0.10 pp	4	88.31%
2 layers (class model)	88.82%	0.09	0.00 pp	4	88.02%
3 hidden layers	88.81%	0.15	-0.01 pp	5	88.12%

Adding or removing a layer made no real difference (both changes are well under the 0.25 pp noise level). A single layer was marginally better and is cheaper to run. The reason is that sentiment in this data is carried mostly by individual words (“excellent”, “worst”), which even a one-layer network picks up; extra layers only add room to memorise.

3.4 Number of neurons per layer (width)

Table 4. Effect of the number of neurons per layer (2 layers)

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
32 neurons	89.04%	0.07	+0.22 pp	4	88.30%
8 neurons	88.96%	0.08	+0.14 pp	5	88.21%
64 neurons	88.83%	0.06	+0.01 pp	4	88.14%
16 neurons (class model)	88.82%	0.09	0.00 pp	4	88.02%
128 neurons	88.73%	0.12	-0.09 pp	4	88.20%

Width was the most useful setting. **32 neurons per layer gave the best result of the entire study (89.04%, +0.22 pp)** and was also very consistent across runs. Going wider did not help: Figure 3 shows accuracy peaking at 32 and falling at 64 and 128. Figure 4 shows why: the wider networks (pink and red lines) start over-training sooner and their error climbs much faster, because extra neurons give the model more capacity to memorise.

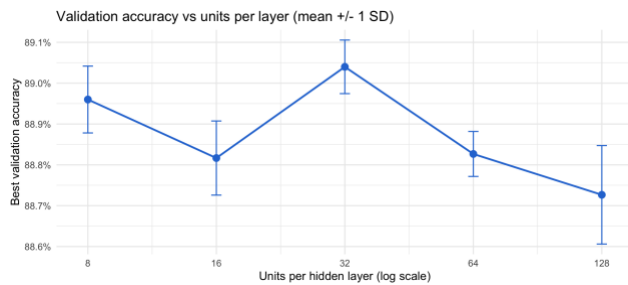


Figure 3. Validation accuracy by neurons per layer; the peak is at 32.

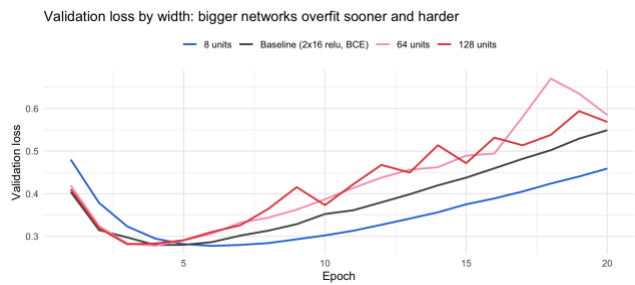


Figure 4. Practice-exam error by width: wider networks over-train sooner and harder.

3.5 Error measure (loss function): mse vs binary cross-entropy

Table 5. Effect of the loss function

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
Binary cross-entropy (class model)	88.82%	0.09	0.00 pp	4	88.02%
Mean squared error (mse)	88.73%	0.12	-0.08 pp	5	88.19%

Switching to mse made **no meaningful difference** (-0.08 pp). Both measures push the model toward the right answer, so the final predictions are similar. Binary cross-entropy remains the better choice for yes/no questions: it penalises confident wrong answers much more heavily, so the model learns fastest from its worst mistakes.

3.6 Neuron rule (activation): tanh vs relu

Table 6. Effect of the activation function

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
relu (class model)	88.82%	0.09	0.00 pp	4	88.02%
tanh	88.80%	0.21	-0.02 pp	4	88.07%

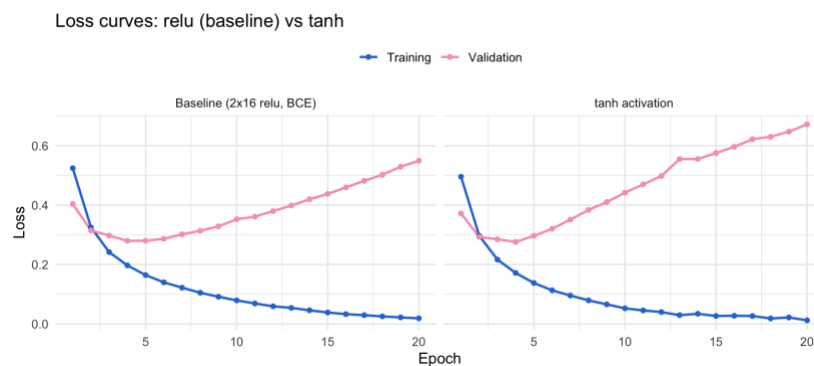


Figure 5. Training (blue) and practice-exam (pink) error for relu vs tanh. tanh over-trains more steeply after epoch 4.

The older tanh rule reached the **same peak accuracy** as relu (-0.02 pp) but was **less reliable**: its results varied more than twice as much between runs (spread 0.21 vs 0.09), and Figure 5 shows its practice-exam error rising faster after epoch 4. relu remains the better default because it is simpler to compute and does not “saturate” (stop learning) for large inputs the way tanh can.

3.7 Anti-memorising techniques (regularization)

Table 7. Effect of regularization (2 layers $\times$ 16 neurons)

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
Dropout 0.3	88.93%	0.11	+0.11 pp	5	88.31%
L2 0.0001 + dropout 0.5	88.89%	0.02	+0.07 pp	7	88.37%
Dropout 0.5	88.83%	0.08	+0.02 pp	7	88.31%
No regularization (class model)	88.82%	0.09	0.00 pp	4	88.02%
L2 = 0.0001	88.80%	0.10	-0.02 pp	3	88.23%
L2 = 0.001	88.73%	0.07	-0.09 pp	5	88.19%

Regularization gave **small but consistent benefits**. Dropout 0.3 scored best on validation (+0.11 pp), every dropout version improved **test** accuracy by about 0.3 points, and L2 + dropout was the **most stable version in the study** (spread only 0.02). Figure 6 shows the main effect: with dropout, the error stays low for longer (the best stopping point moves from epoch 4 to 7), and strong L2 (blue) keeps the error curve much flatter. In short, regularization makes the model more forgiving about when training stops, and lets larger networks be used safely.

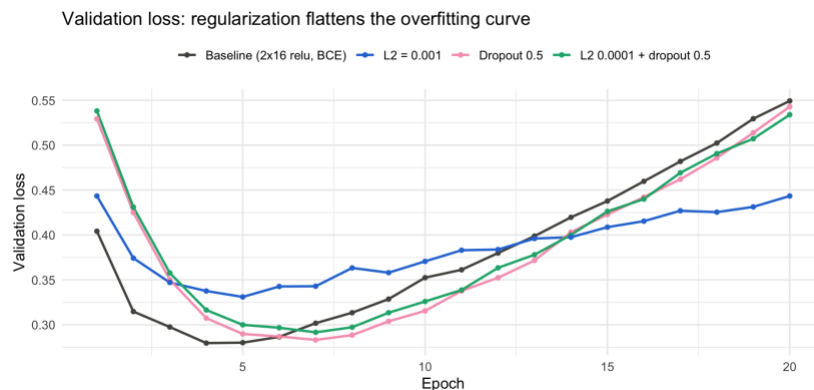


Figure 6. Practice-exam error with and without regularization. Dropout delays over-training; L2 = 0.001 flattens the curve.

3.8 Combining the best ideas and choosing the final model

Table 8. Combined versions

Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Best epoch	Test accuracy
32 neurons + dropout 0.5	89.02%	0.20	+0.20 pp	5	88.33%
32 neurons + dropout 0.5 + L2	88.91%	0.14	+0.09 pp	5	88.37%
64 neurons + dropout 0.5	88.86%	0.24	+0.04 pp	4	88.16%
Class model	88.82%	0.09	0.00 pp	4	88.02%
64 neurons + dropout 0.5 + L2	88.81%	0.22	-0.01 pp	4	88.05%

Combining wider layers with dropout produced 89.02% for 32 neurons, essentially tied with plain 32 neurons (89.04%), while the 64-neuron combinations did not beat the class model. Following the rule set in advance (**choose the highest validation accuracy, never the test score**), the final model is **two layers of 32 neurons with relu and binary cross-entropy**. Figure 7 compares its learning curves with the class model: it reaches a slightly higher practice-exam peak at epoch 4, and like every version it needs to be stopped early.

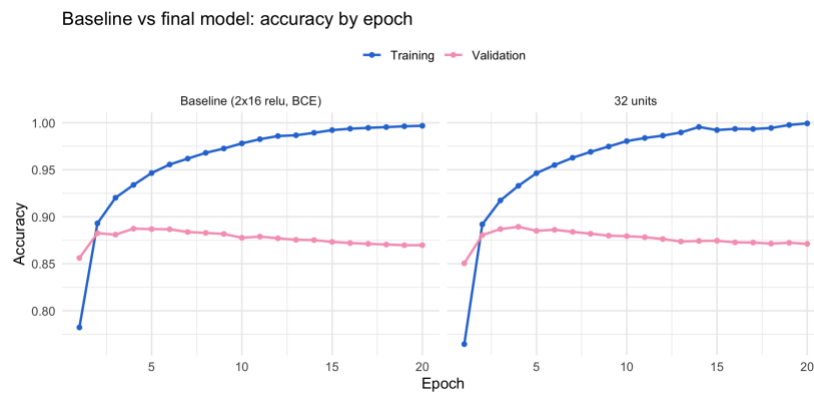


Figure 7. Accuracy by epoch for the class model and the final model (32 neurons). Blue = training reviews, pink = practice exam.

Table 9. Final exam: accuracy on the 25,000 test reviews after retraining on all 25,000 building reviews (average of 3 runs)

Model	Epochs	Test accuracy	Spread ($\pm$)
Class model (2×16 neurons)	4	88.34%	0.25
Final model (2×32 neurons)	4	88.27%	0.33

On the final exam, both models scored about **88.3%**; the 0.07-point gap is smaller than the run-to-run spread, so they are statistically tied. This is an honest and useful result: the small validation advantage of the wider model **does not translate into a reliable real-world gain**, which confirms the central message that the class model was already near the limit of this approach. Because the test reviews were never used to make any choice, this $\approx 88.3\%$ is a trustworthy estimate of how the model will perform on new reviews.

4. Conclusions

- **The model design is not the bottleneck.** All 18 versions scored between 88.7% and 89.1%. Layers, error measure and activation rule barely matter for this problem.
- **Moderate width helps; excessive width hurts.** 32 neurons per layer was best; 128 neurons over-trained and scored lowest.
- **When to stop training matters most.** Every version over-trained after about 4–5 epochs; automatic stopping at the best point is essential.
- **Regularization buys stability, not a higher ceiling.** Dropout and L2 made results steadier and slightly better on unseen reviews.
- **The $\approx 89\%$ ceiling comes from the input, not the network.** Because the model only sees which words are present, it cannot understand phrases such as “not good” or sarcasm.

5. Recommendations

1. **Adopt the final configuration:** two hidden layers of 32 neurons, relu activation, a sigmoid output and binary cross-entropy loss.
2. **Always use automatic early stopping** based on validation accuracy (about 4 epochs here) instead of a fixed training length.
3. **Add dropout (0.3–0.5) whenever the network is made larger**, to control over-training and improve stability.
4. **Keep the defaults** (relu, binary cross-entropy); tanh and mse offer no benefit.

- Invest next in a richer input representation (models that read word order, such as word embeddings, recurrent networks or Transformers) rather than further tuning of this network.

6. Limitations and Next Steps

Three runs per version can only separate differences larger than about 0.25 pp; smaller differences should be read as ties. Choosing both the stopping point and the best version on the same practice exam makes validation scores slightly optimistic, which is why the untouched test score is reported. Design choices were mostly varied one at a time, so some combinations (for example a tuned learning rate) were not explored. Finally, the word-presence input ignores word order and frequency, which limits accuracy to about 89%; sequence-reading models are the natural next step.

Appendix: Complete Results

Table 10. All 18 versions (validation used for every decision; test shown for reference only)

Group	Setting tested	Validation accuracy	Spread ($\pm$)	Change vs baseline	Test accuracy
Baseline	Class model (2 $\times$ 16, relu, BCE)	88.82%	0.09	0.00 pp	88.02%
A. Depth	1 hidden layer	88.91%	0.13	+0.10 pp	88.31%
A. Depth	3 hidden layers	88.81%	0.15	-0.01 pp	88.12%
B. Width	32 neurons (final model)	89.04%	0.07	+0.22 pp	88.30%
B. Width	8 neurons	88.96%	0.08	+0.14 pp	88.21%
B. Width	64 neurons	88.83%	0.06	+0.01 pp	88.14%
B. Width	128 neurons	88.73%	0.12	-0.09 pp	88.20%
C. Loss	mse loss	88.73%	0.12	-0.08 pp	88.19%
D. Activation	tanh	88.80%	0.21	-0.02 pp	88.07%
E. Regularization	Dropout 0.3	88.93%	0.11	+0.11 pp	88.31%
E. Regularization	L2 0.0001 + dropout 0.5	88.89%	0.02	+0.07 pp	88.37%
E. Regularization	Dropout 0.5	88.83%	0.08	+0.02 pp	88.31%
E. Regularization	L2 = 0.0001	88.80%	0.10	-0.02 pp	88.23%
E. Regularization	L2 = 0.001	88.73%	0.07	-0.09 pp	88.19%
F. Combined	32 neurons + dropout 0.5	89.02%	0.20	+0.20 pp	88.33%
F. Combined	32 neurons + dropout 0.5 + L2	88.91%	0.14	+0.09 pp	88.37%
F. Combined	64 neurons + dropout 0.5	88.86%	0.24	+0.04 pp	88.16%
F. Combined	64 neurons + dropout 0.5 + L2	88.81%	0.22	-0.01 pp	88.05%

Where to find the code: the complete R Markdown code (Keras 3 / TensorFlow), its knitted HTML output with every graph and table, and the saved results file are in the GitHub repository https://github.com/smoham32/smoham32_ADML_64061, folder “Assignment 1”. Code is deliberately left out of this report so that the findings are easy to follow.